설비 고장을 예측하는 AI 데이터 분석

함수 · 함수의 기본

함수의 개념과 구조 → 함수 호출과 이름 → 코드 중복과 함수화

posco

CONTENTS

목차

01 함수의 개념과 구조

02 함수 호출과 이름

03 코드 중복과 함수화

posco

posco

01

함수의 개념과 구조

함수란 무엇인가 · def 함수의 기본 구조

posco

반복되는 코드의 문제

복사-붙여넣기를 두 번 이상 하고 있다면, 그건 코드가 잘못 가고 있다는 신호다.

SIGNAL 01

함수가 필요하다는 신호

똑같은 코드를 여러 번 쓰는 순간이 함수가 필요하다는 신호

SIGNAL 02

가장 자주 마주칠 판단 기준

복사-붙여넣기를 두 번 이상 한다면 코드가 잘못 가는 신호

posco

반복 코드가 부르는 세 가지 문제

같은 코드가 화면을 가득 채우는 순간, 길이·수정·실수 세 문제가 동시에 터진다.

01 · 코드 길이

화면이 가득 차 흐름이 안 보인다

판정 코드를 다섯 번 복사하면

스물다섯 줄로 늘어남

설비가 스무 개면 백 줄이 된다.
같은 코드가 화면을 채우면
전체 흐름을 파악하기 어려워진다.

가독성과 흐름 파악이 **무너진다**

02 · 수정 지옥

기준 하나 바꾸기가 일이 된다

기준 하나 바꾸려면

복사한 모든 자리를 일일이 수정

80도를 75도로 바꿀 때 한 자리만 빠뜨려도
80도·75도 판정이 섞이는 버그가 생긴다.

하나라도 빠뜨리면 **버그**

03 · 실수 발생

조용히 틀린 결과를 낸다

복사 중 이름 하나만 안 바꿔도

엉뚱한 결과 출력

펌프2를 점검하는 코드가
엉뚱하게 펌프1 결과를 출력한다.
멈추지 않고 조용히 틀린다.

가장 잡기 어려운 **버그 유형**

posco

함수란 무엇인가

함수는 특정 작업을 하는 코드를 묶고 이름을 붙인 것이다.

DEFINITION

함수의 정의

특정 작업을 하는 코드를 묶고 이름을 붙인 것

여러 줄의 코드를 한 묶음으로 보관해 두고,
필요할 때마다 그 이름을 부르기만 하면 된다.

HOW IT WORKS

이름을 부르면 묶어 둔 코드가 실행

이름을 부르면 묶어 둔 코드 전체가 한 번에 실행

print, len, type, int — 우리는 이미 함수를 써 왔다.
복잡한 작업이 이름 하나에 들어 있다.

posco

내장 함수와 사용자 정의 함수

누군가 미리 만들어 둔 함수를 쓰는 단계에서, 내가 직접 만드는 단계로 한 발 나아간다.

01 · 내장 함수

기본 제공되어 바로 쓰는 함수

print, len처럼 기본 제공되어 바로 쓰는 함수

print, len, type, int, input — 모두 누군가 미리 만들어 둔 내장 함수.
우리는 가져다 썼을 뿐.

02 · 사용자 정의

내가 직접 만들어 이름을 붙임

내가 직접 만들어 이름을 붙이는 함수

설비 온도 판정 함수, 센서값 평균 함수처럼 내 일에 필요한 도구를 직접 만들어 쓴다.

함수를 비유로 이해하기

함수는 입력을 넣으면 정해진 일을 하고 결과를 내주는 기계.

METAPHOR 01

자판기 — 동전 → 음료

동전을 넣고 버튼을 누르면 음료가 나오듯

함수도 넣으면 정해진 작업을 거쳐 결과가 나온다.

자판기 안에서 무슨 일이 일어나는지 몰라도 된다 — 입력과 출력만 알면 충분하다.

METAPHOR 02

믹서기 — 사과 → 사과주스

사과를 넣으면 사과주스, 딸기를 넣으면 딸기주스

넣는 값에 따라 결과가 달라진다.

믹서기에 넣는 재료가 매개변수, 나오는 결과가 반환값 — 뒤에서 배운다.

함수를 보는 두 가지 관점

한 번 잘 만들어 두면, 그 다음부터는 안을 신경 쓰지 않고 이름만 부르면 된다 — 함수의 가장 큰 힘.

01 · 만드는 사람

자판기 회로를 설계하는 사람

함수 안에서 일어날 일을 코드로 설계

함수 안에서 어떤 일이 일어날지를 직접 코드로 짠다.
회로를 설계하는 입장.

02 · 쓰는 사람

버튼을 누르는 사람

안의 원리는 몰라도 이름만 부르면 됨

print를 쓸 때 그 안의 복잡한 코드를 몰라도 됐던 것처럼,
호출하는 입장에서는 이름만 알면 된다.

posco

함수를 쓰면 달라지는 세 가지

짧아짐 → 수정 쉬움 → 재사용 — 함수를 쓰면 코드가 이렇게 달라진다.

STEP 1

짧아짐

반복 코드가 함수 정의와 호출로 줄어든다.
설비 다섯 개 판정도
함수 이름 다섯 번 부르기로 끝난다.

전체 흐름을 한눈에 파악

STEP 2

수정 쉬움

함수 안 한 곳만 고치면
모든 호출 자리에 반영된다.
복사한 자리를 빠뜨리는 실수가 사라진다.

실무에서 가장 크게 와닿는 차이

STEP 3

재사용

한 번 만든 함수를 다른 프로그램에서도 호출.
온도 판정 함수를 다음 주 보고서 프로그램에서도
그대로 부른다.

분석가의 자산이 쌓인다

posco

함수의 핵심 원칙

한 번 만들고 여러 번 쓴다 — 함수의 원칙을 코드에서 실현하는 도구.

PRINCIPLE 01

함수의 본질

한 번 만들고 여러 번 쓴다

설계는 한 번, 호출은 여러 번.
시간을 아끼는 가장 단순한 규칙.

함수의 **존재 이유**

PRINCIPLE 02

데이터 분석에서 특히

같은 처리를 반복 적용

같은 전처리·계산을 여러 데이터에
반복 적용할 일이 정말 많다.

분석가의 **일상**

PRINCIPLE 03

잘 쓰는 사람과 못 쓰는 사람의 차이

분석가의 필수 도구

함수를 잘 쓰는 사람과 못 쓰는 사람의 코드 차이가
매우 크다.

실력 차이가 **드러나는 자리**

posco

def 함수의 다섯 부분 — 앞 세 가지

기호가 많아 보여도 다섯 부분으로 나눠 보면 단순하다 — 먼저 앞 세 가지부터.

01 · DEF

define — 정의한다의 줄임말

함수를 정의하겠다는 키워드

```
def
```

Python에게 보내는 **선언 신호**

02 · 함수 이름

내가 직접 정한다

나중에 부를 때 쓰는 이름

```
greet
```

호출할 때 부르는 **간판**

03 · 괄호

입력값을 받는 자리

입력값을 받는 자리, 지금은 빈 상태

```
()
```

매개변수 자리는 뒤에서

posco

def 함수의 다섯 부분 — 뒤 두 가지

콜론과 들여쓰기 — 빠뜨리기 쉬우니 특히 주의할 두 부분.

04 · 콜론

if, for, while과 같은 원리

함수 내용이 시작된다는 표시

```
def greet():
```

여기서부터 함수 내용이 시작된다는 표시.
조건문에서 if 뒤에 콜론을 찍었던 것과 같은 원리.

05 · 들여쓰기

보통 스페이스 네 칸

함수에 속한 코드임을 나타냄

```
····print("...")
```

들여쓰기된 코드들이 모두 이 함수에 속한다는 뜻.
콜론과 들여쓰기는 빠뜨리기 쉬우니 특히 주의.

def 함수의 기본 구조

함수를 만들기만 해서는 아무 일도 일어나지 않는다 — 따로 호출해야 비로소 실행된다.

CODE

```
def greet():
    print("점검을 시작합니다")
    print("장비를 착용하세요")
```

첫 줄에 **def 함수이름 괄호 콜론**, 둘째 줄부터 들여쓰기해서 할 일을 적는다. 함수 안의 코드는 위에서 아래로 적은 순서대로 실행된다.

정의만으로는 실행되지 않는다

함수를 만든 것은 레시피를 적어 둔 것일 뿐 — 작동시키려면 따로 불러야 한다.

STATE 01

함수 정의 직후

레시피를 적어 둔 것일 뿐

함수를 정의한 코드를 실행해도 화면에는 아무것도 출력되지 않는다.
레시피를 적어 서랍에 넣어 둔 상태일 뿐이다.

STATE 02

실제로 작동시키려면

따로 불러야 하며 이것이 호출

서랍에서 레시피를 꺼내 실제 요리를 시작하는 단계.
호출이라는 동작이 함수를 작동시킨다.

콜론과 들여쓰기의 역할

초보자가 함수를 만들 때 가장 많이 내는 에러 두 가지 — 콜론 누락과 들여쓰기 실수.

요소	역할	빠뜨리면
콜론 `:`	함수 시작 신호 if·for·while과 같은 원리	**SyntaxError** **문장이 아직 안 끝났다고 판단**
들여쓰기	코드 소속 표시 다른 언어는 중괄호, Python은 들여쓰기로만	**IndentationError** **코드 소속 판단 불가**

미리 제대로 알아 두면 에러를 크게 줄일 수 있다. 조건문이나 반복문에서 콜론을 빼먹거나 들여쓰기 때문에 에러를 본 경험을 떠올려 보자.

posco

들여쓰기는 Python의 문법

들여쓰기는 보기 좋음의 문제가 아니라 문법 그 자체 — Python은 칸으로 코드를 묶는다.

01 · 콜론

여기서부터 함수의 내용

시작 신호

"여기서부터 시작이다"라고 Python에게 알리는 표시.

02 · 들여쓰기

이 코드가 어디에 속하는가

소속 표시

"이 코드는 위 함수에 속한다"는 정보를 칸 수로 표현한다.

03 · 둘다

빠뜨리면 함수가 안 만들어진다

함수의 뼈대

Python은 칸으로 코드를 묶는 언어.

두 가지 모두 함수의 뼈대를 이룬다.

posco

함수 블록의 범위

들여쓰기 수준이 곧 함수의 경계선이다.

CODE

```
def greet():
    print("점검 시작")    # 함수 안
print("프로그램 종료")    # 함수 밖
```

들여쓴 **점검 시작**은 함수 안, 들여쓰기 없는 **프로그램 종료**는 함수 밖. 함수 밖 코드는 함수를 부르지 않아도 그냥 실행된다.

함수 범위가 정하는 것

함수 블록의 범위는 함수를 부를 때만 실행되는 코드의 경계.

01 · 경계

호출할 때만 실행되는 코드

함수를 부를 때만 실행되는 코드의 경계

함수 블록 안에 있는 코드는
호출되지 않으면 절대 실행되지 않는다.

02 · 안과 밖

위치에 따라 실행 시점이 다름

안에 있으면 호출 시점,
밖에 있으면 흐름이 닿을 때

함수 안 코드는 호출할 때만,
밖 코드는 위에서 아래로 흐름이 그 줄에 닿을 때 실행.

03 · 끝의 기준

들여쓰기가 사라지는 줄

함수의 끝을 정하는 것은 오직 들여쓰기 수준

마지막 한 줄의 들여쓰기를 빠뜨리면
그 줄은 함수 밖이 되어 프로그램 시작하자마자 출력된다.

함수를 호출하는 방법

괄호가 핵심 — 이름만 쓰고 괄호를 안 붙이면 실행되지 않는다.

CODE

```
def greet():
    print("점검을 시작합니다")
greet()   # 함수 이름 + 괄호 = 호출
```

함수 이름을 적고 그 뒤에 **괄호**를 붙이면 호출된다. greet 괄호를 쓰는 순간 함수 안의 코드가 위에서부터 차례로 실행된다.

02

함수 호출과 이름

정의 시점과 호출 시점 · 함수 이름 짓기 규칙

posco

괄호는 실행 버튼

한 번 만든 함수는 몇 번이든 호출할 수 있다 — greet 괄호를 세 번 쓰면 함수가 세 번 실행된다.

WITHOUT ()

이름만 쓰면

이름만 쓴 greet는 함수가 있다는 사실만 가리킴

```
greet
```

"이런 함수가 있다"고 가리키기만 할 뿐 실행은 일어나지 않는다.

WITH ()

괄호를 붙이면

괄호를 붙인 greet 괄호라야 실행 명령

```
greet()
```

괄호는 "지금 실행하라"는 버튼이다. 몇 번이든 누를 수 있다.

posco

정의 시점과 호출 시점

만드는 시점과 부르는 시점은 다르고, 반드시 만든 다음에 불러야 한다.

MOMENT 01 · 정의 시점

def로 만드는 순간

기억만 하고 실행 안 함

Python은 "이런 이름의 함수가 있다"는 사실만 기억한다.
레시피를 적어 서랍에 넣어 둔 상태.

MOMENT 02 · 호출 시점

괄호로 부르는 순간

함수 코드가 실제로 실행

서랍에서 레시피를 꺼내 요리를 시작하는 순간.
함수 안 코드가 실제로 실행되는 건 오직 이 호출 시점이다.

posco

정의보다 호출이 먼저 오면

위쪽에 함수들을 먼저 정의해 두고, 아래쪽에서 호출하는 구조로 짠다.

CODE

```
greet()   # 아직 모르는 이름 → NameError
def greet():
    print("점검 시작")
```

호출이 정의보다 먼저 오면 Python은 **NameError**를 낸다. 서랍에 아직 넣지도 않은 레시피를 꺼내려는 것과 같다.

posco

함수 실행 순서 따라가기

각주를 보러 잠깐 내려갔다가 다시 본문으로 돌아오는 것과 같다.

STEP 1

시작

프로그램이 맨 위에서 시작해
위에서 아래로 코드를 읽어 내려간다.

위에서 시작

→

STEP 2

호출

함수 호출을 만나면
흐름이 잠깐 함수 안으로 점프한다.

함수로 점프

→

STEP 3

함수 안

함수 안 코드를 위에서부터
순서대로 모두 실행한다.

코드 실행

→

STEP 4

복귀

함수 안 코드가 다 끝나면
원래 호출했던 자리로 돌아온다.

자리로 복귀

→

STEP 5

끝

복귀 자리의 다음 줄부터 이어 실행,
코드 끝에 도달하면 종료.

다음 줄

posco

실행 흐름 세 단계

손가락으로 코드 위를 움직이며 점프와 복귀를 짚어 보면 흐름이 또렷이 보인다.

STEP 1

def는 기억만

위쪽의 def는 그냥 함수를 기억만 하고 지나간다.
함수 안 코드는 아직 실행되지 않는다.

이름만 등록

→

STEP 2

호출 → 진입

함수 호출을 만나면 함수 안으로 들어가
함수 안의 코드가 실제로 실행된다.

함수 안으로 점프

→

STEP 3

종료 → 복귀

함수가 끝나면 호출한 자리로 돌아와
그 다음 코드가 이어진다.
손가락으로 짚어 보면 또렷하다.

원래 자리로

posco

실습 1. 첫 함수 만들고 호출하기

def로 함수를 만들고 괄호로 호출해 실행을 확인하기

단계

① def와 함수 이름, 괄호, 콜론으로 함수를 정의
② 함수 안 코드를 네 칸 들여써서 실행할 내용 작성
③ 함수 정의 아래에서 이름에 괄호를 붙여 호출
④ 여러 번 호출해 매번 실행되는지 확인

예상 결과

점검을 시작합니다
(두 번 호출하면 두 번 출력)

posco

함수 이름 짓기 규칙

규칙을 어기면 SyntaxError가 나서 실행조차 안 된다.

01 · 시작 문자

변수 이름 규칙과 동일

영문자나 밑줄로 시작, 숫자로 시작 불가

숫자로 시작하면 에러가 난다. 변수 이름과 같은 규칙.

check_temp (가능), **2nd_check** (불가)

02 · 사용 문자

snake_case 표기법

영문자 숫자 밑줄만, 띄어쓰기 불가

두 단어를 이으려면 밑줄로 연결한다.
check_temperature 처럼.

모두 소문자에 밑줄로 잇는 **스네이크 표기법**

03 · 예약어 회피

Python이 이미 쓰고 있는 단어

print, if, for 등은 이름으로 못 씀

Python이 이미 쓰고 있는 단어를 함수 이름으로 쓰면
혼란이 생긴다.

어기면 **SyntaxError** — 실행조차 안 됨

함수 이름 가능 여부

숫자 시작 금지, 띄어쓰기 금지, 내장 함수 이름 겹침 주의 — 세 가지를 기억한다.

이름	가능	이유
`check_temp`	가능	소문자에 밑줄 — 표준 스네이크 표기법
`print_data`	가능	동사로 시작 — 좋은 이름의 조건도 만족
`2nd_check`	불가	숫자로 시작 — Python은 이를 허용하지 않음
`check temp`	불가	띄어쓰기 — 두 단어는 밑줄로 이어야 함

posco

좋은 함수 이름의 조건

동작 · 구체성 · 일관성 — 세 가지를 모두 만족하는 이름이 좋은 이름이다.

01 · 동작 표현

함수는 어떤 작업을 수행한다

동사로 시작해 하는 일을 드러냄

온도를 확인하는 함수라면 check_temperature 처럼.

`check_, get_, print_, calc_`

02 · 구체성

무엇을 다루는지까지

무엇을 다루는지까지 이름에 담음

process는 무엇을 처리하는지 알 수 없다.
process_sensor_data라면 명확하다.

`process_sensor_data`

03 · 일관성

비슷한 함수는 비슷한 방식으로

비슷한 함수는 비슷한 방식으로 명명

온도 확인을 check_temp로 지었다면 압력 확인은
check_pressure로 짓는 식.

`check_temp · check_pressure`

이름만 보고 알 수 있게

이름만 잘 지어도 코드 읽는 속도가 크게 달라진다.

READABILITY 01

함수 안을 안 봐도

함수 안을 안 봐도 무슨 일을 하는지

며칠 뒤 다시 열어 봐도, 이름이 명확하면
안을 들여다보지 않아도 무슨 함수인지 바로 알 수 있다.

READABILITY 02

주석을 대신한다

이름을 잘 지으면 주석 없이도 코드가 읽힘

데이터 분석에서는 함수가 수십 개씩 쌓인다.
이름만 잘 지어도 코드 읽는 속도가 크게 달라진다.

함수를 여러 번 호출하기

함수의 진짜 힘은 한 번 만들어 여러 번 쓰는 데 있다.

CODE

```
def print_line():
    print("=" * 20)
print_line()
print_line()
```

한 번 정의한 print_line을 **두 번 부르면 구분선이 두 번 출력**. 부르는 횟수에 제한이 없고, 부를 때마다 함수 안 코드가 처음부터 끝까지 다시 실행된다.

매번 새로 실행된다

함수를 부를 때마다 매번 깨끗한 상태에서 새로 시작한다.

01 · 호출 횟수

부르고 싶은 만큼

제한 없음

한 번 만든 함수는
한 줄에서 다섯 줄까지든, 천 줄까지든 부를 수 있다.

02 · 시작 상태

깨끗한 상태로 다시

매번 처음부터 실행

새로 호출하면 함수는 처음부터 다시 실행된다.

03 · 독립성

이전 호출과 분리됨

호출끼리 독립적

이전 호출에서 무슨 일이 있었든 영향 없음.
결과가 꼬이지 않는다.

한 줄 함수와 여러 줄 함수

줄 수가 아닌, 그 함수가 하나의 명확한 작업을 담당하는가가 중요하다.

구분	한 줄 함수	여러 줄 함수
예시	구분선 출력	점검 안내
줄 수	1줄	여러 줄
적합	단순 작업	여러 단계
공통	**한 작업**	**한 작업**

줄 수가 많고 적음보다, 그 함수가 하나의 명확한 작업을 담당하는가가 중요. 표 맨 아래 줄을 보세요.

posco

한 함수는 한 가지 일

좋은 함수의 기준 — 한 함수는 한 가지 일만 하는 게 좋다.

ANTI-PATTERN

여러 일을 하는 함수

한 함수가 여러 일을 하면 이름 짓기도 수정도 어려움

점검 안내도 하고 데이터 저장도 하고 보고서도 출력하는 함수는
이름 짓기도 어렵고 수정하기도 어렵다.

BEST PRACTICE

여러 함수로 나누기

작업이 여러 개라면 함수를 여러 개로 나누는 게 나음

한 함수당 한 가지 일. 안내 함수, 저장 함수, 보고서 함수로 나누면
각각 이름도 명확하고 수정도 쉽다.

posco

03

코드 중복과 함수화

코드 중복을 찾는 눈 · 리팩토링의 개념

코드 중복을 찾는 눈

함수를 잘 쓰려면 먼저 "여기는 함수로 만들면 좋겠다"는 자리를 알아보는 눈이 필요하다.

SIGNAL 01

복붙 흔적

값 몇 개만 다른 코드 덩어리가 여러 번 등장

압축기A 온도 출력, 펌프1 온도 출력이 이름만 바뀐 채 줄줄이 이어진다면 명백한 중복.

SIGNAL 02

반복 처리

반복문 안의 처리가 길고 복잡한 경우

반복문도 중복을 줄이는 도구지만,
반복되는 처리 자체가 길고 복잡하면 그 처리를 함수로 빼는 게 깔끔하다.

posco

중복 코드의 모습

값만 다르고 구조가 똑같은 코드 덩어리 — 함수화의 가장 강한 신호.

CODE

```
print("압축기A 온도 확인 중")
print("결과를 기록합니다")
print("펌프1 온도 확인 중")
print("결과를 기록합니다")   # ← 같은 줄 반복!
```

"결과를 기록합니다"가 **반복**되고 있다. 데이터 분석에서는 같은 전처리, 같은 계산을 여러 데이터에 반복하는 일이 정말 많다.

중복 코드를 함수로 묶는 절차

찾기 → 묶기 → 교체 → 확인 — 정해진 순서를 따르면 실수 없이 깔끔하게 정리할 수 있다.

STEP 1

찾기

반복되는 공통 부분을 식별한다.
값만 다른 같은 구조를 찾는다.

공통 부분 식별

STEP 2

묶기

그 부분을 def로 함수로 만들어 묶는다.
이름도 함께 정한다.

def로 정의

STEP 3

교체

원래 반복되던 자리를 모두 함수 호출로 바꾼다.

호출로 변경

STEP 4

확인

실행해서 결과가 이전과 같은지 검증.
코드 모양은 바뀌었지만 동작은 그대로여야 한다.

결과 검증

결과 확인을 빠뜨리지 않기

함수로 묶기 전과 후의 출력이 완전히 같아야 한다 — 한 글자라도 다르면 무언가 놓친 것.

RULE 01

함수로 묶은 뒤 반드시

실행해 결과가 이전과 같은지 확인

묶기 전 코드의 출력을 먼저 확인해 두고, 묶은 후에 다시 실행해 비교한다.

RULE 02

동작이 달라졌다면

묶는 과정에서 무언가 빠뜨린 것

출력이 한 글자라도 다르면 무언가 놓친 것.

묶는 과정에서 빠뜨린 줄이나 순서를 다시 확인해야 한다.

리팩토링의 개념

리팩토링은 동작은 그대로 두고 구조만 더 좋게 바꾸는 일.

DEFINITION

Refactoring — 정식 용어

동작은 그대로, 구조만 더 좋게

사용자가 보기에는 프로그램이 똑같이 작동하지만, 코드 내부는 더 깔끔해진 상태.

EXAMPLE

대표적인 리팩토링

중복 코드를 함수로 묶는 것

함수로 묶기 전과 후의 출력은 완전히 같지만 코드는 짧아지고 수정하기 쉬워졌다.

먼저 동작하게 그다음 깔끔하게

코드는 한 번 짜고 끝나지 않으므로 리팩토링이 중요하다.

01 · 일단 돌아가게

탐색하듯 코드를 짠다

처음에는 일단 돌아가게 짠다

데이터 분석에서는 탐색하듯 코드를 짜다 보면
중복이 쌓이기 쉽다.

02 · 작동 확인 후

중복을 함수로 정리

작동이 확인되면 중복을 함수로 정리

어느 정도 작동이 확인되면
중복을 함수로 정리하는 리팩토링을 해 둔다.

03 · 전후 검증

같은 입력, 같은 결과

전후로 같은 입력에 같은 결과가 나오는지

리팩토링 전후로 같은 입력에 같은 결과가 나오는지
확인하는 습관이 필요하다.

실습 2. 반복 코드를 함수로 정리하기

여러 번 반복되는 코드를 함수로 묶어 호출로 대체하기

단계

① 여러 설비마다 반복되는 안내 여러 줄을 확인
② 그 줄들을 def로 함수 하나에 묶기
③ 반복하던 자리를 함수 호출로 대체
④ 실행해 이전과 같은 출력이 나오는지 확인

예상 결과

각 설비마다 같은 안내 3줄이 함수 호출로 출력

posco

함수 호출 결과 예측하기

실행 전 결과를 예측해 보고 맞는지 확인하는 것 — 코드 이해의 핵심 습관.

CODE

```
def hello():
    print("안녕하세요")
hello()
hello()   # 안녕하세요는 몇 번 나올까?
```

hello를 두 번 부르면 **"안녕하세요"가 두 번 출력**. 실행 전에 머릿속으로 먼저 예측해 보자.

예측 후 확인하는 습관

실행 전에 먼저 그려보기 → 호출만 따라가기 → 차이에서 배우기.

01 · 머릿속 그리기

실행 전 시뮬레이션

무엇이 어떤 순서로 출력될지 머릿속으로

실행하기 전에 무엇이 어떤 순서로 출력될지
머릿속으로 먼저 그려본다.

02 · 정의는 건너뛰기

호출만 따라간다

함수 정의는 건너뛰고 호출만 따라가기

함수 정의는 건너뛰고, 호출을 만나면
함수 안 출력을 확인하고,
끝나면 다음 줄로.

03 · 차이에서 배우기

예측 vs 실제

예측과 실제가 같으면 이해,
다르면 차이에서 배움

예측이 맞으면 흐름을 제대로 이해한 것,
다르면 그 차이에서 배움이 일어난다.

예측이 틀릴 때가 기회

예측 습관은 나중에 오류를 찾는 디버깅 능력으로 이어진다.

PREDICT 01

맞췄을 때

예측이 맞으면 이해가 정확하다는 확인

내가 코드를 제대로 이해하고 있다는 신호.
다음 코드에 대한 자신감으로 이어진다.

PREDICT 02

틀렸을 때 — 가장 값진 순간

예측이 틀리면 어디가 어긋났는지 정확히

내가 잘못 알고 있던 부분이 드러나는 가장 값진 순간.
이 습관은 디버깅 능력으로 이어진다.

posco

함수와 자판기 모델

함수는 입력, 처리, 출력의 세 부분을 가진 기계.

INPUT

입력

함수에 넣어 주는 값.

자판기로 치면 동전.

매개변수

뒤에서 배움

→

PROCESS

처리 (지금까지)

함수 안에서 일어나는 작업.

자판기로 치면 음료 준비.

지금까지 우리가 만든 함수는 이 부분만 있었다.

함수 본체

def 안의 코드

→

OUTPUT

출력

함수가 내놓는 결과.

자판기로 치면 음료가 나오는 것.

반환값

뒤에서 배움

앞으로 배울 입력과 출력

이 둘을 배우면 같은 함수로 넣는 값에 따라 다른 결과를 만들 수 있다.

함수에 값을 넣어 주는 방법

매개변수

함수에 값을 넣어 주는 방법.
자판기에 동전을 넣듯이 함수에 입력을 전달한다.

함수가 결과를 돌려주는 방법

반환값

함수가 결과를 돌려주는 방법.
믹서기에 사과를 넣으면 사과주스, 딸기를 넣으면 딸기주스가 나오듯.

함수가 없을 때의 코드 비교

문구 수정에서 가장 큰 차이가 난다 — 데이터 분석 실무에서 가장 크게 와닿는 지점.

항목	함수 없음	함수 사용
코드 길이	길어짐	짧아짐
문구 수정	모든 자리	한 곳만
실수 위험	높음	낮음
재사용	어려움	쉬움

> 함수가 없으면 같은 안내 두 줄을 세 번 복사해 여섯 줄. 함수를 쓰면 안내 두 줄을 함수 안에 한 번만 적고 세 번 호출.

posco

코드가 커질수록 벌어지는 차이

함수를 쓰는 사람과 안 쓰는 사람의 생산성이 완전히 달라진다.

SMALL PROGRAM

작은 프로그램일 때

함수 이점이 잘 안 느껴질 수 있음

수업에서 예로 든 짧은 코드에서는 함수의 이점이 잘 안 느껴질 수 있다.

LARGE PROGRAM

실무 데이터 분석에서

길고 자주 수정하는 작업에서 생산성이 완전히 달라짐

코드가 커질수록 기하급수적으로 벌어진다.

데이터 분석처럼 길고 자주 수정하는 작업에서 함수는 결정적 도구.

실습 3. 함수 실행 흐름 추적하기

정의와 호출의 순서를 따라가며 실행 흐름 예측하기

단계

① 함수를 정의만 하면 실행되지 않음을 확인
② 호출 줄에서 함수 안으로 들어갔다 나오는 흐름 따라가기
③ 여러 번 호출할 때 출력이 몇 번 나올지 먼저 예측
④ 실행해 예측과 맞는지 확인

예상 결과

정의만 하면 출력 없음
hello() 두 번 → 안녕하세요 두 번

실습 4. 함수로 설비 점검 자동화하기

여러 함수를 정의하고 순서대로 호출해 점검 흐름을 자동화하기

단계

① 구분선을 출력하는 함수를 정의
② 점검 안내 여러 줄을 출력하는 함수를 정의
③ 두 함수를 설비마다 순서대로 호출
④ 실행해 각 설비마다 같은 안내가 반복되는지 확인

예상 결과

구분선과 점검 안내 2줄이 설비마다 반복 출력